

SWINGBOT C++

Documentation technique des classes

Architecture DIP — C++20 — Google Test

Couche	Classes	Rôle
Interfaces	IDataFeed, IBroker, IStrategy, IRiskManager, ILogger, IIndicator	Abstractions DIP — zéro dépendance concrète
Modèles	Bar, Signal, Order, Position, Account, BotState, SwingConfig	Structures de données pures
Indicateurs	EMA, RSI, CrossoverDetector	Calcul technique — implémentent IIndicator
Stratégie	SwingStrategy	Logique EMA 9/21 + RSI — implémente IStrategy
Risk Mgmt	RiskManager	Sizing, stop-loss, take-profit, trailing stop
Loggers	ConsoleLogger, FileLogger, CompositeLogger, NullLogger	Logging multi-cible
Bot	TradingBot	Orchestrateur principal — injection de dépendances
Tests	MockDataFeed, MockBroker	Doublures de test — isolent le bot du réseau

1. Modèles de données

Les modèles sont des structures de données pures (POD) sans logique métier. Ils sont définis dans `include/models/Models.hpp` et utilisés par toutes les couches du système.

struct

Bar

models/Models.hpp

Représente une bougie OHLCV (Open/High/Low/Close/Volume) sur une période donnée.

Méthode / Attribut	Type retour	Description
date	string	Date de la bougie (ex: "2024-01-15")
open / high / low / close	double	Prix d'ouverture, haut, bas, clôture
volume	long	Volume d'échanges de la période

struct

Signal

models/Models.hpp

Résultat d'évaluation d'une stratégie. Contient le type de signal et le contexte associé.

Méthode / Attribut	Type retour	Description
type	SignalType	BUY, SELL, ou HOLD
symbol	string	Symbole de l'ETF (ex: "QQQ")
price	double	Prix au moment du signal
reason	string	Explication textuelle du signal
isBuy() / isSell() / isHold()	bool	Accesseurs pratiques pour le type

struct

Order

models/Models.hpp

Représente un ordre soumis au broker avec son statut d'exécution.

Méthode / Attribut	Type retour	Description
symbol	string	Symbole tradé
side	OrderSide	BUY ou SELL
quantity	int	Nombre d'actions
status	OrderStatus	PENDING, FILLED, CANCELLED, REJECTED
orderId	string	Identifiant unique de l'ordre

struct

Position

models/Models.hpp

Position ouverte sur un symbole chez le broker.

Méthode / Attribut	Type retour	Description
shares	int	Nombre d'actions détenues
avgPrice	double	Prix moyen d'achat
unrealizedPnl	double	Profit/Perte non réalisé en dollars

struct

Account

models/Models.hpp

État du compte de trading (cash disponible, valeur totale, statut).

Méthode / Attribut	Type retour	Description
cash	double	Liquidités disponibles pour trader
equity	double	Valeur totale du compte (cash + positions)
status	string	"ACTIVE", "SUSPENDED", etc.

struct

BotState

models/Models.hpp

État interne persistant du bot entre deux cycles d'exécution.

Méthode / Attribut	Type retour	Description
inPosition	bool	True si une position est ouverte
buyPrice	double	Prix d'achat de la position courante
peakPrice	double	Prix le plus haut atteint depuis l'achat (trailing stop)
holdDays	int	Nombre de jours depuis l'achat

Paramètres de configuration de la stratégie swing. Modifie ce struct pour ajuster le comportement du bot.

Méthode / Attribut	Type retour	Description
symbol	string	ETF cible (défaut: "QQQ")
emaFast / emaSlow	int	Périodes EMA rapide/lente (défaut: 9/21)
rsiPeriod	int	Période RSI (défaut: 14)
rsiBuyMax	double	RSI max pour entrer en position (défaut: 55.0)
rsiSellMin	double	RSI min pour déclencher une vente (défaut: 70.0)
stopLossPct	double	Stop-loss en % (défaut: 0.05 = 5%)
takeProfitPct	double	Take-profit en % (défaut: 0.10 = 10%)
trailingStopPct	double	Trailing stop depuis le pic (défaut: 0.03 = 3%)
minHoldDays	int	Jours minimum avant de vendre (défaut: 3)

2. Interfaces (Dependency Inversion)

Le cœur de l'architecture DIP. Toutes les dépendances du **TradingBot** passent par ces interfaces. Cela permet de substituer n'importe quelle implémentation (broker réel, mock de test, autre stratégie) sans modifier le bot.



ILogger

core/Interfaces.hpp

Abstraction du système de logging. Permet d'avoir plusieurs destinations (console, fichier, null pour tests) sans coupler le code à une implémentation.

Méthode / Attribut	Type retour	Description
info(msg)	void	Message informatif
warn(msg)	void	Avertissement non-bloquant
error(msg)	void	Erreur – situation anormale
debug(msg)	void	Détail de débogage (verbose)



IDataFeed

core/Interfaces.hpp

Source de données marché. Peut être Alpaca, Yahoo Finance, ou un mock de test. Le bot ne sait pas d'où viennent les données.

Méthode / Attribut	Type retour	Description
getBars(symbol, days)	vector<Bar>	Barres journalières historiques
getLatestPrice(symbol)	optional<double>	Prix temps réel (nulopt si indisponible)
isMarketOpen()	bool	Vérifie si la bourse est ouverte



IBroker

core/Interfaces.hpp

Exécution des ordres de trading. Peut être Alpaca paper, Alpaca live, ou un mock. Retourne optional<> pour gérer les rejets proprement.

Méthode / Attribut	Type retour	Description
submitBuy(symbol, qty)	optional<Order>	Soumet un ordre d'achat marché
submitSell(symbol, qty)	optional<Order>	Soumet un ordre de vente marché
getPosition(symbol)	optional<Position>	Position actuelle (nulopt si aucune)
getAccount()	Account	État du compte (cash, equity, statut)



IStrategy

core/Interfaces.hpp

Logique de génération de signaux. Indépendante du broker et du data feed. Prend des barres historiques, retourne un signal.

Méthode / Attribut	Type retour	Description
evaluate(bars)	Signal	Analyse les barres et génère BUY/SELL/HOLD
name()	string	Identifiant de la stratégie pour le logging



IRiskManager

core/Interfaces.hpp

Gestion du risque — calcul du sizing et validation des conditions de sortie. Séparé de la stratégie pour respecter le principe de responsabilité unique (SRP).

Méthode / Attribut	Type retour	Description
positionSize(capital, price, sl, risk)	int	Calcule le nombre d'actions selon le risque dollar
isTradeAllowed(account, position)	bool	Vérifie si un trade est autorisé
checkExitConditions(...)	optional<string>	Retourne la raison de sortie ou nullopt



IIndicator

core/Interfaces.hpp

Interface template générique pour les indicateurs techniques. T est le type de la valeur retournée (double pour EMA/RSI). Permet d'injecter n'importe quel indicateur dans la stratégie.

Méthode / Attribut	Type retour	Description
compute(prices)	vector<T>	Calcule l'indicateur sur une série de prix
name()	string	Nom de l'indicateur (ex: "EMA(9)")

3. Indicateurs techniques

Implémentations concrètes de `IIndicator<double>`. Header-only dans `include/indicators/Indicators.hpp`. Chaque indicateur est stateless — aucune donnée mémorisée entre les appels.

class

EMA

indicators/Indicators.hpp

Exponential Moving Average. Donne plus de poids aux prix récents qu'une SMA simple. Formule: $EMA(t) = \text{Prix}(t) \times k + EMA(t-1) \times (1 - k)$ où $k = 2 / (\text{période} + 1)$. Le seed initial est une SMA sur les 'période' premières valeurs.

Méthode / Attribut	Type retour	Description
EMA(period)	—	Constructeur — lève <code>invalid_argument</code> si <code>period <= 0</code>
compute(prices)	vector<double>	Retourne vide si pas assez de données
name()	string	Retourne "EMA(N)" où N est la période
period()	int	Accesseur de la période

Complexité: $O(n)$ en temps, $O(n)$ en espace.

class

RSI

indicators/Indicators.hpp

Relative Strength Index — oscillateur entre 0 et 100. Mesure la vitesse et l'amplitude des mouvements de prix. $RSI > 70$ = suracheté (signal de vente potentiel), $RSI < 30$ = survendu. Utilise le smoothing de Wilder (EMA spéciale avec facteur $1/\text{période}$).

Méthode / Attribut	Type retour	Description
RSI(period=14)	—	Constructeur — lève <code>invalid_argument</code> si <code>period <= 1</code>
compute(prices)	vector<double>	Index 0 à <code>period-1</code> = 0.0, calcul commence à l'index <code>period</code>
name()	string	Retourne "RSI(N)"

<>

CrossoverDetector

indicators/Indicators.hpp

Utilitaire stateless qui détecte les croisements entre deux séries (EMA rapide vs EMA lente). Un croisement BULLISH se produit quand fast passe au-dessus de slow, signalant une tendance haussière potentielle.

Méthode / Attribut	Type retour	Description
detect(fast, slow)	Cross	Analyse les 2 dernières valeurs. Retourne BULLISH, BEARISH

4. Stratégie de trading

class

SwingStrategy

strategies/SwingStrategy.hpp

Implémentation concrète de IStrategy. Combine le croisement EMA 9/21 et le RSI 14 pour générer des signaux de swing trading sur horizon 3-15 jours. Utilise le pattern Factory Method pour la création.

Logique d'entrée (BUY) — 3 conditions simultanées:

Croisement EMA	EMA rapide croise au-dessus de l'EMA lente (CrossoverDetector::BULLISH)
RSI modéré	RSI < rsiBuyMax (défaut 55) — le marché n'est pas encore suracheté
Prix > EMAs	Prix de clôture > EMA rapide ET > EMA lente — confirmation de tendance

Logique de sortie (SELL) — 1 condition suffit:

Croisement baissier	EMA rapide croise en dessous de l'EMA lente
RSI suracheté	RSI > rsiSellMin (défaut 70) — momentum extrême

Méthode / Attribut	Type retour	Description
SwingStrategy(config, emaFast, emaSlow, rsi)	—	Constructeur — injection des indicateurs via IIndicator
create(config) [static]	unique_ptr<SwingStrategy>	Factory method — crée avec EMA et RSI par défaut
evaluate(bars)	Signal	Calcule les indicateurs et retourne BUY/SELL/HOLD
name()	string	Retourne "SwingStrategy_EMA_RSI"
config()	const SwingConfig&	Accesseur de la configuration

5. Gestion du risque

class

RiskManager

bot/RiskManager.hpp

Implémentation de IRiskManager. Responsable du sizing des positions et de la surveillance des conditions de sortie (stop-loss, take-profit, trailing stop). Séparé de la stratégie pour respecter le Single Responsibility Principle.

Formule de sizing (Risk-Based Position Sizing):

$$\text{shares} = (\text{Capital} \times \text{RiskPct}) / (\text{Prix} \times \text{StopLossPct})$$

Exemple: capital=\$10,000 | risk=2% | prix=\$420 | stop=5% → risque_dollar=\$200 | risque_par_action=\$21 → 9 actions. Garantit que la perte maximale est toujours de 2% du capital si le stop-loss est déclenché.

Conditions de sortie (priorité décroissante):

1. Stop-Loss	currentPrice <= buyPrice × (1 - stopLossPct)	Protection prioritaire — s'active immédiatement
2. Take-Profit	currentPrice >= buyPrice × (1 + takeProfitPct)	Sécurise les gains à l'objectif
3. Trailing Stop	currentPrice <= peakPrice × (1 - trailingStopPct) + holdDays >= minHoldDays	Protège les gains latents après le délai minimum

Méthode / Attribut	Type retour	Description
RiskManager(maxCapitalUsagePct=0.95)	–	Constructeur – limite l'usage du capital à 95% max
positionSize(capital, price, sl, risk)	int	Retourne 0 si capital=0 ou paramètres invalides
isTradeAllowed(account, position)	bool	False si déjà en position, compte inactif, ou cash < \$10
checkExitConditions(...)	optional<string>	Retourne la raison de sortie ou nulopt si OK

6. Système de logging

Quatre implémentations de ILogger pour différents contextes. Toutes sont thread-safe grâce à std::mutex. Le CompositeLogger permet de combiner plusieurs destinations.

class

ConsoleLogger

bot/Logger.hpp

Logger vers stdout avec couleurs ANSI. INFO=vert, WARN=jaune, ERROR=rouge, DEBUG=cyan. Mode verbose optionnel pour les messages debug.

Méthode / Attribut	Type retour	Description
info/warn/error/debug(msg)	void	Affiche avec timestamp et couleur
setVerbose(bool)	void	Active/désactive les messages DEBUG

class

FileLogger

bot/Logger.hpp

Logger vers un fichier texte. Ouvre en mode append — les logs survivent aux redémarrages du bot. Même format que ConsoleLogger sans les codes couleur.

Méthode / Attribut	Type retour	Description
FileLogger(path)	—	Ouvre le fichier en append (crée si inexistant)
info/warn/error/debug(msg)	void	Écrit avec timestamp et flush immédiat

class

CompositeLogger

bot/Logger.hpp

Délègue à plusieurs loggers en simultané. Exemple d'usage: console + fichier en même temps. Pattern Composite.

Méthode / Attribut	Type retour	Description
addLogger(shared_ptr<ILogger>)	void	Ajoute un logger à la liste
info/warn/error/debug(msg)	void	Appelle chaque logger dans l'ordre d'ajout

class

NullLogger

bot/Logger.hpp

Logger qui ne fait rien — /dev/null du logging. Utilisé dans les tests unitaires pour supprimer tout output et garder les résultats propres.

Méthode / Attribut	Type retour	Description
info/warn/error/debug(msg)	void	No-op – corps vide

7. TradingBot — Orchestrateur

class

TradingBot

bot/TradingBot.hpp

Classe centrale du système. Orchestre tous les composants via leurs interfaces. Ne connaît aucune implémentation concrète — tout est injecté via le constructeur (DIP). Contient la machine à états de la position (BotState).

Flux d'exécution de runOnce():

Étape	Composant	Action
1	IDataFeed	isMarketOpen() → sortie immédiate si fermé
2	IDataFeed	getBars(symbol, 60) → récupère l'historique
3	IStrategy	evaluate(bars) → Signal (BUY/SELL/HOLD)
4	IBroker	getPosition() → position actuelle
5	IRiskManager	checkExitConditions() → stop-loss / take-profit / trailing
6	IBroker	submitSell() si condition de sortie détectée
7	IRiskManager	isTradeAllowed() + positionSize() si signal BUY
8	IBroker	submitBuy() si trade autorisé

Méthode / Attribut	Type retour	Description
TradingBot(dataFeed, broker, strategy, riskMgr, logger)	–	Constructeur – injection de toutes les dépendances
runOnce()	void	Exécute un cycle complet de trading (testable unitairement)
run(intervalSec=86400)	void	Boucle infinie – vérifie toutes les N secondes
stop()	void	Arrête la boucle run() proprement
state()	const BotState&	Accesseur de l'état interne (pour les tests)
setState(state)	void	Modifie l'état interne (pour les tests)
config()	const SwingConfig&	Accesseur de la configuration de la stratégie

8. Mocks de test

Implémentations de test de IDataFeed et IBroker. Permettent de tester le TradingBot sans connexion réseau ni compte Alpaca. Inclus uniquement dans les builds de test — jamais en production.

class

MockDataFeed

bot/Mocks.hpp

Simule une source de données marché. Tu configures les barres à retourner et l'état du marché avant chaque test.

Méthode / Attribut	Type retour	Description
setBars(vector<Bar>)	void	Configure les barres retournées par getBars()
setLatestPrice(double)	void	Configure le prix spot retourné
setMarketOpen(bool)	void	Simule marché ouvert ou fermé
buildTrend(start, days, change) [static]	vector<Bar>	Génère une série de barres avec tendance constante
buildBullishCrossover(n) [static]	vector<Bar>	Génère des barres avec croisement EMA haussier

class

MockBroker

bot/Mocks.hpp

Simule l'exécution des ordres. Enregistre tous les ordres soumis pour vérification dans les assertions de test. Peut simuler le rejet d'ordres.

Méthode / Attribut	Type retour	Description
setAccount(Account)	void	Configure l'état du compte retourné
setPosition(optional<Position>)	void	Configure la position retournée
setRejectOrders(bool)	void	Si true, submitBuy/Sell retournent nullopt
buyCount()	int	Nombre d'ordres BUY soumis (pour assertions)
sellCount()	int	Nombre d'ordres SELL soumis (pour assertions)
orders()	vector<OrderRecord>	& Historique complet des ordres soumis
clearOrders()	void	Réinitialise l'historique des ordres